

# The SPSC Lock-Free Ring Buffer

A First-Principles Treatment of Memory Ordering, Cached-Index Optimization,  
and Unsigned-Overflow Correctness

Robert (Bo) Hou · Robert's Technical Notes · June 2026

---

**Abstract.** The single-producer single-consumer (SPSC) lock-free ring buffer is the canonical data backbone of low-latency systems. This report develops the design from first principles in three layers. First, we derive the *minimal* memory ordering required on each atomic operation, establishing a precise criterion for when an *acquire* load is mandatory versus when a *relaxed* load suffices — a distinction routinely over-specified in practice. We show that the monotonicity argument establishes *value safety* but not the *happens-before* relation C++ requires, and that slot reuse therefore forces *acquire* on the capacity-cursor load. Second, we analyze the cached-index optimization, proving its safety via a monotonicity argument and clarifying that it reduces *coherence cost*, not correctness or fill rate. Third, we give a formal congruence-theoretic proof that the absolute-counter design remains correct under unsigned hardware overflow, strengthened with an invariant-maintenance induction.

---

## 1. The Baseline Design and Notation

We store trivially-copyable elements of type  $T$  in a fixed array of size  $Capacity$ , a power of two. Two monotonically increasing absolute counters — `tail_` (written only by the producer) and `head_` (written only by the consumer) — index the buffer through a bitwise mask.

```

template <typename T, size_t Capacity>
class SpscQueue {
    static_assert((Capacity & (Capacity - 1)) == 0,
        "Capacity must be power of 2");

    alignas(64) std::atomic<size_t> head_{0}; // consumer writes
    alignas(64) std::atomic<size_t> tail_{0}; // producer writes
    alignas(64) std::array<T, Capacity> buf_;

public:
    bool try_push(const T& item) { // producer only
        size_t t = tail_.load(std::memory_order_relaxed); // [A]
        size_t h = head_.load(std::memory_order_acquire); // [B]
        if (t - h == Capacity) return false; // full
        buf_[t & (Capacity - 1)] = item;
        tail_.store(t + 1, std::memory_order_release); // [C]
        return true;
    }

    bool try_pop(T& out) { // consumer only
        size_t h = head_.load(std::memory_order_relaxed); // [D]
        size_t t = tail_.load(std::memory_order_acquire); // [E]
        if (h == t) return false; // empty
        out = buf_[h & (Capacity - 1)];
        head_.store(h + 1, std::memory_order_release); // [F]
        return true;
    }
};

```

Three structural decisions deserve emphasis up front.

- **Absolute counters, not wrapped indices.** `head_` and `tail_` only ever increase; the physical slot is counter & (Capacity-1). This makes the occupancy `t - h` a single subtraction, with no branch to handle wrap.
- **Power-of-two capacity.** The mask & (Capacity-1) is exactly mod Capacity, eliminating a division.
- **Per-line alignment.** `head_` and `tail_` occupy separate cache lines (`alignas(64)`) to avoid false sharing: the producer writes only `tail_`, the consumer only `head_`; co-locating them would make the line ping-pong between cores under MESI even though the two variables are logically independent.

## 2. Memory Ordering: The Minimal Specification

### 2.1 Two independent facts: atomicity vs. ordering

It is essential to separate two guarantees that `std::atomic` provides.

- **Atomicity:** an atomic load/store is indivisible (no torn reads), and for any single atomic variable all threads observe one consistent *modification order* — even under *relaxed*.
- **Ordering:** whether operations on *different* variables become visible to another thread in a particular order. This is what `memory_order` controls.

A *relaxed* operation gives the first but not the second. This is why reading one's own counter is always safe with *relaxed*: by read-your-own-write plus single-variable modification-order consistency, the producer's relaxed load of `tail_ [A]` and the consumer's relaxed load of `head_ [D]` always observe their own latest store.

## 2.2 The release/acquire handshake

- **release** (the publisher): on the store that announces “ready,” all writes sequenced before it are guaranteed visible to any thread that later reads the announced value. A one-way downward fence — prior writes cannot sink below it.
- **acquire** (the reader): on the load that reads the announced value, all reads sequenced after it cannot be hoisted above it. A one-way upward fence.

The handshake fires *only* when the acquire load actually reads the value written by the release store. At that instant a *synchronizes-with* edge is established, and everything the publisher wrote before its release becomes visible to the reader after its acquire.

**Remark.** Release/acquire need not be “symmetric.” A release store may be paired with a relaxed load and remain legal; the relaxed load simply does not enjoy the synchronization. Whether a load needs acquire depends on what the loading thread does *next*, not on whether the opposite store is release.

## 2.3 The easy load: [E]

**[E] — consumer loads `tail_`: must be acquire.** After the load, the consumer executes `out = buf[h]`, reading a slot the producer filled via `buf[t] = item` before its release store [C]. Crucially, the address `buf[h]` depends on `h` (the consumer's own counter), *not* on the loaded `t`. Therefore no data dependency forces the payload read to wait for the `tail_` load; only a *control* dependency through `if (h == t)` stands between them, and control dependencies do not prevent speculative execution. A CPU may speculatively execute `out = buf[h]` before the load resolves, reading stale or partially-written data. The acquire on [E] forbids this reordering; paired with [C]'s release, it guarantees that once the new `t` is observed, `buf[h]` is already fully written. So far the criterion is intuitive: the loader subsequently reads payload the other thread published, so it must synchronize.

## 2.4 The hard load [B]: three levels of understanding

Site [B] — the producer's load of `head_` — is where the genuinely interesting reasoning lives, and the honest way to present it is as a climb through three levels of understanding. At each level a different amount of knowledge is in hand, and at each level a different answer “wins.” The punchline is that the answer travels **acquire → relaxed → acquire**: you start forced into acquire, a clever insight tempts you down to relaxed, and the language standard drives you back up to acquire. Tracing that arc — not just landing on the final answer — is what separates understanding the machine from reciting a rule.

## Level 1 — Before you know the monotonicity trick: acquire wins

Suppose you reason only from the structure of the code, with no special insight. You look at the push path and try to justify the cheapest possible ordering, relaxed, by appealing to *dependencies*:

```
size_t h = head_.load(/* ? */);    // [B]
if (t - h == Capacity) return false;
buf_[t & (Capacity - 1)] = item;    // the write
```

The tempting argument runs: “Line 2 uses `h`, and line 3 is guarded by line 2, so the write is chained behind the load of `head_` and cannot be reordered before it — therefore relaxed is enough; the hardware will preserve the dependency.” This feels airtight: modern CPUs never violate a true data dependency (RAW), so why pay for a fence?

**The argument is wrong, and seeing why is the first level of understanding.** The write’s address is `t & mask`, which depends on `t` — *not* on the loaded `h`. So there is no true data dependency from the write back to the load. The only thing connecting them is the `if (t - h == Capacity)` branch — a *control* dependency. And control dependencies do not constrain a modern out-of-order CPU: it will *speculate* the branch (bet that it is not full) and execute the write early, before the load has even resolved. The dependency you thought would pin the ordering does not exist; the branch you leaned on is jumped over by speculation.

So at Level 1 the “use relaxed because of the dependency” shortcut collapses. With nothing else in hand, you cannot prove relaxed is safe, and the conservative, correct choice is to synchronize: **acquire wins**. Note that this is already the *right* answer — it just is not yet proven to be the *only* answer. We have not invoked any clever property of the queue. Acquire is simply the default a careful engineer reaches for when nothing licenses anything weaker.

## Level 2 — Once you know the monotonicity trick: relaxed wins

Now you bring in a genuinely clever observation, the kind that separates someone who has thought hard about the data structure from someone who has not: `head_` **and** `tail_` **are monotone — they only ever increase** (a detail many engineers never notice; they are absolute counters, never decremented, never reset, wrapping only on integer overflow, which Section 4 proves harmless). From monotonicity follows a clean safety proof:

**Proposition (Value safety under relaxed [B]).** Because `head_` only increases, a relaxed load can only return a value  $\leq$  the true current `head_`. The producer therefore *underestimates* the free space. The worst that can happen is a *spurious full* — the producer refuses to push when a slot was in fact free — which is harmless and self-corrects on the next call. The dangerous opposite, a *spurious free* (believing a slot is available when it is not, which would overwrite live data), is impossible. Hence the producer never computes an out-of-bounds index and never logically overwrites an unread slot.

This proof is *correct*, and it is beautiful. In a purely algorithmic, state-machine sense, monotonicity buys back exactly the safety that the missing dependency failed to provide at Level 1. If you were building this FIFO in hardware — say a Verilog FIFO on an FPGA, where the only question is whether the logic is sound — you could legitimately drop the fence and save the gates. At the end of Level 2, relaxed looks not merely acceptable but *optimal*: provably safe, and one synchronization cheaper. **Relaxed wins.** This is the level at which a very strong engineer, reasoning correctly about the algorithm, confidently writes relaxed — and is about to be ambushed by the language.

### Level 3 — Once you know the C++ standard: acquire wins again

The ambush is that C++ does not grade your algorithm. The compiler judges by the C++ memory model, whose definition of a data race is purely *structural* and indifferent to your cleverness:

**C++ data race (informal).** If two threads access the same *non-atomic* object, at least one access is a write, and the two are not ordered by a happens-before relation, the program's behavior is **undefined** — *regardless of physical timing*.

To see the trap, follow a single slot `buf_[i]` as the ring wraps and the slot is **reused**. Four events span the two threads:

```
Consumer (cycle 1):          Producer (cycle 2):
(1) out = buf_[i];           (3) h = head_.load(/* ? */); // [B]
(2) head_.store(release); // [F] (4) buf_[i] = item;        // overwrite
```

Event (1) and event (4) touch the *same non-atomic* `buf_[i]`: the consumer reads it, the producer later overwrites it. One is a write. For this to be race-free, the read (1) must *happens-before* the overwrite (4). Within each thread the ordering is free: (1) is sequenced-before (2), and (3) is sequenced-before (4). The missing link is the *cross-thread* edge between (2) and (3) — and that edge exists **only if** [B] reads the release value of [F] with acquire, forming a synchronizes-with.

This is exactly the **[F] release  $\leftrightarrow$  [B] acquire** pairing. It is a *reverse* synchronization edge — it runs **from the consumer to the producer**: “the consumer has finished reading `buf_[i]` *happens-before* the producer may overwrite it.” It is the mirror image of the forward edge **[C] release  $\leftrightarrow$  [E] acquire**, which runs from producer to consumer (“the producer has

written `buf[t]` *happens-before* the consumer may read it”). The two edges point in opposite directions and connect different pairs of operations; together they let every slot rotate safely between the two threads. Neither edge is decoration: the forward edge stops the consumer from reading data the producer has not finished writing; the reverse edge stops the producer from overwriting data the consumer has not finished reading.

Now the verdict. If [B] is relaxed, the producer still *numerically* observes the new `head_` (relaxed respects each variable’s modification order), so in physical time it really does wait — the monotonicity proof of Level 2 is not lying about the hardware. But the C++ abstract machine records **no** synchronizes-with edge between (2) and (3). With no edge, (1) does not happens-before (4); two threads access the non-atomic `buf[i]` with one write and no ordering — a data race — and the program is **undefined behavior**. Once it is UB, the compiler is free under `-O3` to reorder, fuse, or delete the `buf_` accesses entirely; the physical-time safety Level 2 so carefully proved simply evaporates, because the standard never promised to honor it. **Acquire wins again** — this time as a hard requirement, not a default.

### 2.4.1 Reconciliation: two theorems, one slogan

The three levels are not in contradiction; they answer questions of increasing subtlety. Level 1 establishes that *dependency intuition* cannot license relaxed (control  $\neq$  data dependency, and speculation jumps the branch). Level 2 establishes *value safety* via monotonicity — the algorithm computes only correct indices. Level 3 establishes that C++ legality is about *happens-before form*, not value safety, and that slot reuse therefore demands the reverse `[F]→[B]` edge. The decisive realization is that **a proof of value safety is not a proof of data-race freedom**: they are different theorems about different things, and lock-free code must win both. Monotonicity proves the values are right; it cannot manufacture the happens-before edge the abstract machine demands.

So the slogan for [B] is: *acquire is the honest default; monotonicity tempts a downgrade to relaxed and even proves it safe in value; but the standard refuses, because value safety is not data-race freedom — so acquire stands*. This is exactly why Folly’s `ProducerConsumerQueue`, `Boost.Lockfree`, and every production SPSC use acquire at this site. The detour through relaxed is not an embarrassment to hide; walking it is what demonstrates command of both the algorithm and the C++ memory model, rather than a memorized “always use acquire.”

## 2.5 Summary of the six orderings

| Site | Operation                      | Order   | Reason                              |
|------|--------------------------------|---------|-------------------------------------|
| [A]  | <code>tail_.load (own)</code>  | relaxed | read-your-own-write                 |
| [B]  | <code>head_.load (peer)</code> | acquire | anchors reverse sync for slot reuse |
| [C]  | <code>tail_.store</code>       | release | publishes <code>buf_</code> payload |

|     |                   |         |                                     |
|-----|-------------------|---------|-------------------------------------|
| [D] | head_.load (own)  | relaxed | read-your-own-write                 |
| [E] | tail_.load (peer) | acquire | payload buf_[h] read afterward      |
| [F] | head_.store       | release | read completes before slot is freed |

There are **two** load-bearing handshakes, one per cursor. The tail chain **[C] release ↔ [E] acquire** carries the forward visibility of the payload (producer writes buf\_[t] → consumer reads buf\_[h]). The head chain **[F] release ↔ [B] acquire** carries the reverse synchronization that makes slot reuse safe (consumer finishes reading buf\_[i] → producer may overwrite it). Dropping either chain's acquire/release breaks one direction of the buf\_ happens-before and admits a data race.

## 2.6 Platform realization

On x86 (TSO), the entire family of relaxed/acquire/release loads and stores compiles to plain mov; the hardware already forbids all reorderings except store→load. Only seq\_cst stores incur extra cost (an xchg or mov+mfence to drain the store buffer), which is why low-latency code avoids seq\_cst. On weakly-ordered architectures (ARM, POWER), acquire/release emit real barrier instructions; all four of [B], [C], [E], [F] then lower to genuine fences, and none may be relaxed without breaking a happens-before chain. The abstraction is portable: the engineer reasons only about data-and-flag happens-before; the compiler lowers it to each platform's correct instructions.

## 3. The Cached-Index Optimization

### 3.1 Motivation: coherence is correct but not free

Every `try_push` that loads `head_` pays a cross-core cost. The consumer has most recently written `head_`, so its cache line is in the Modified state on the consumer's core. When the producer reads it, MESI must transfer ownership across cores — tens to hundreds of cycles — and the subsequent consumer write reclaims the line, producing a ping-pong. MESI guarantees the producer reads the *latest* value, but reading it is expensive. The optimization removes most of these cross-core reads.

### 3.2 Mechanism

The producer keeps a private, non-atomic `cached_head_`. Most pushes test fullness against this cached value, touching only the producer's own L1. Only when the cached value indicates “full” does the producer perform a real `head_.load()` to refresh.

```
bool try_push(const T& item) {
    size_t t = tail_.load(std::memory_order_relaxed);
    if (t - cached_head_ == Capacity) { // cheap local test
        cached_head_ = head_.load(std::memory_order_acquire); // refresh
        if (t - cached_head_ == Capacity)
            return false; // genuinely full
    }
    buf_[t & (Capacity - 1)] = item;
    tail_.store(t + 1, std::memory_order_release);
    return true;
}
```

### 3.3 Safety

**Lemma (Cache monotonicity).** At all times  $\text{cached\_head\_} \leq \text{actual\_head}$ , where `actual_head` is the current value of `head_`.

*Proof.* `cached_head_` is only ever assigned from a past load of `head_`. Since the consumer increments `head_` monotonically, any previously observed value is  $\leq$  the current one.  $\square$

**Proposition (Conservative correctness).** If the producer writes a slot, that slot is genuinely free; the producer never overwrites unread data.

*Proof.* The producer writes only when its fullness test fails, i.e. when (after a possible refresh)  $t - \text{cached\_head\_} < \text{Capacity}$ . By the monotonicity lemma,  $t - \text{actual\_head} \leq t - \text{cached\_head\_} < \text{Capacity}$ . Thus the true occupancy is strictly below capacity, so a free slot exists. The cached value can only cause a *spurious full* (a conservative refusal), never a *spurious free*; “false full” is harmless (it triggers one refresh), while “false free” — which would corrupt — cannot occur.  $\square$

### 3.4 What it does *not* do

The optimization does not throttle or control the producer's fill rate, nor does it govern how fast the buffer fills. The fill rate is determined solely by the relative throughput of producer and consumer. The cached index changes only the *cost of the fullness check* — replacing a frequent cross-core load with an occasional one. The producer is ultimately gated by the physical capacity bound, not by the cache. By analogy: the cached index is a cheaper *gauge* for reading the water level; switching gauges does not change how fast water flows in or out.

## 4. Correctness Under Unsigned Hardware Overflow

This section formalizes why the absolute-counter design remains correct when the  $k$ -bit counters overflow and wrap. We first establish the invariant the algorithm maintains, then prove addressing and distance invariance under that invariant.

### 4.1 Formal setting

Let  $\mathbb{Z}_{2^k}$  be the ring of integers modulo  $2^k$  (a  $k$ -bit unsigned register,  $k = 64$  typically). Unsigned addition and subtraction are  $a \oplus b \equiv a + b \pmod{2^k}$  and  $a \ominus b \equiv a - b \pmod{2^k}$ . Let  $N = 2^m$  with  $0 < m < k$  be the capacity, and let the addressing map  $f : \mathbb{Z}_{2^k} \rightarrow \mathbb{Z}_N$  be the bitwise truncation  $f(x) = x \text{ AND } (2^m - 1)$ .

**Lemma (Modular homomorphism of  $f$ ).** Since the mask  $(2^m - 1)$  isolates the low  $m$  bits,  $f(x) \equiv x \pmod{2^m}$ . Because  $2^m \mid 2^k$ , the map descends through the quotient and preserves additive structure:  $f(a \oplus b) \equiv (a + b \pmod{2^k}) \pmod{2^m} \equiv (a + b) \pmod{2^m} \equiv (f(a) + f(b)) \pmod{2^m}$ .

### 4.2 The maintained invariant

Let  $t, h \in \mathbb{Z}_{2^k}$  be the producer and consumer absolute counters.

**Invariant (No-overflow occupancy).** At every reachable state,  $0 \leq (t \ominus h) < N$ .

All subsequent correctness rests on this invariant, so we prove the algorithm maintains it, rather than assuming it.

**Theorem 1 (Invariant maintenance).** Under single-producer single-consumer operation, the invariant  $0 \leq t \ominus h < N$  holds in every reachable state.

*Proof.* By induction over operations.

**Base case.** Initially  $t = h = 0$ , so  $t \ominus h = 0$ , and  $0 \leq 0 < N$ .

**Inductive step.** Assume  $0 \leq t \ominus h < N$ . Two operations can change the state.

**Push.** The producer advances  $t \mapsto t \oplus 1$  only after its guard verifies  $t \ominus h < N$ , i.e.  $t \ominus h \leq N - 1$ . After the increment the occupancy becomes  $(t \oplus 1) \ominus h = (t \ominus h) + 1 \leq (N - 1) + 1 = N$ . Combined with the lower bound, the new occupancy lies in  $[1, N]$ , within the admissible

range. (At the extremal value  $N$  the buffer is exactly full, and the next push guard correctly rejects.) Only the producer modifies  $t$ , so  $h$  is unchanged and the lower bound  $0 \leq t \ominus h$  is preserved.

**Pop.** The consumer advances  $h \mapsto h \oplus 1$  only after verifying  $t \ominus h > 0$  (non-empty). After the increment,  $t \ominus (h \oplus 1) = (t \ominus h) - 1 \geq 0$ , preserving the lower bound; the upper bound is preserved since occupancy strictly decreases. Only the consumer modifies  $h$ , so  $t$  is unchanged.

Because the producer modifies only  $t$  and the consumer only  $h$ , the two updates never race on the same counter, and each preserves both bounds. Hence the invariant holds in all reachable states.  $\square$

### 4.3 Addressing invariance

**Theorem 2 (Seamless addressing across overflow).** When the producer counter  $t_1 = 2^k - 1$  increments to  $t_2 = t_1 \oplus 1 = 0$ , the physical index remains sequential:  $f(t_2) = (f(t_1) + 1) \bmod N$ .

*Proof. Left side.*  $t_2 = 0$ , so  $f(t_2) = f(0) \equiv 0 \pmod{2^m}$ , and since  $0 \in [0, 2^m - 1]$ ,  $f(t_2) = 0$ .

**Right side.**  $(f(t_1) + 1) \bmod 2^m \equiv (t_1 + 1) \bmod 2^m \equiv (2^k - 1 + 1) \bmod 2^m \equiv 2^k \bmod 2^m$ . Because  $2^m \mid 2^k$ , this is 0.

Both sides equal 0, so by transitivity  $f(t_2) = (f(t_1) + 1) \bmod N = 0$ . The wrap of the absolute counter produces no discontinuity in the physical slot sequence.  $\square$

### 4.4 Distance invariance

**Theorem 3 (Distance invariance via ring arithmetic).** Suppose the producer has wrapped so its nominal value is below the consumer's:  $t_2 = 0 < h$ , with  $h = 2^k - d$  for the true physical distance  $0 < d < N$ . Then the unsigned subtraction yields the exact distance with no branch:  $t_2 \ominus h = d$ .

*Proof.* In  $\mathbb{Z}_{2^k}$ , subtraction is addition of the additive inverse:  $t_2 \ominus h \equiv (t_2 + 2^k - h) \bmod 2^k = (0 + 2^k - (2^k - d)) \bmod 2^k = d \bmod 2^k$ . By the maintained invariant,  $0 < d < N < 2^k$ , so  $d$  lies in the fundamental domain and the outer reduction is the identity:  $d \bmod 2^k = d$ . Hence  $t_2 \ominus h = d$ .  $\square$

**Remark (Two's complement is the implementation, not the definition).** Because  $t, h$  are unsigned, the operative algebra is ring arithmetic modulo  $2^k$ . The term  $2^k - h$  is the additive inverse of  $h$  in this ring; its bit pattern happens to coincide with the two's-complement encoding ( $\text{NOT } h + 1$ ), which is why a single unsigned subtraction on the ALU produces the correct distance even when  $t < h$  numerically. The hardware corrects the apparent “temporal inversion” for free.

## 4.5 Worked bitstream trace

Let  $k = 8$  ( $2^k = 256$ ) and  $N = 8$ . Suppose  $h = 250$  (1111 1010) and the producer wraps to  $t = 0$  (0000 0000); the true distance is  $256 - 250 = 6$ .

- **Additive inverse of  $h$ :** NOT 1111 1010 + 1 = 0000 0101 + 1 = 0000 0110 (value 6).
- **Unsigned add  $t + (2^k - h)$ :** 0000 0000 + 0000 0110 = 0000 0110 (value 6).
- **Mask  $\& (N-1)$ :**  $\& 0000 0111$ : 0000 0110 AND 0000 0111 = 0000 0110 (slot 6).

The cyclic topology of modular arithmetic absorbs the wrap with zero branches and zero misprediction penalty.

## 5. Synthesis

The SPSC ring buffer rewards reasoning at the right level of abstraction — and punishes conflating two notions of correctness. *Value safety* (the producer never computes a bad index) follows from cursor monotonicity, but C++ legality is a question of *happens-before form*: every pair of concurrent non-atomic accesses to `buf_` must be ordered by a synchronizes-with edge, or the program has a data race regardless of physical timing. This forces acquire/release on *both* cursor chains — forward (payload publication) and reverse (slot reuse) — which is why every production implementation does the same. Correctness of *addressing*, by contrast, reduces to two modular facts: the mask is a homomorphism, and unsigned subtraction is ring subtraction. The deep hardware mechanisms — store buffers, cache coherence, speculative execution — explain *why* the abstractions are sound, but the working engineer reasons in terms of happens-before edges, monotone counters, and congruences; and crucially, an algorithmic proof of value safety is *not* a proof of data-race freedom. That distinction is the level at which the code is both written and defended.